

SMART CITY PUBLIC TRANSIT DELAY FORECASTING FOR URBAN AND RURAL AREAS

A Supervised Machine Learning Mini Project Using Python

Prepared as a college mini-project / GitHub submission

Abstract

Public transit reliability is an important smart-city problem because delays influence passenger waiting time, route connectivity, fleet utilisation and the accessibility of essential services. This project presents a supervised machine-learning prototype for forecasting the level of public transit delay in urban and rural settings. The model uses temporal, operational, environmental and traffic-related variables and predicts one of three delay categories: On Time/Minor, Moderate Delay, or Major Delay. A Logistic Regression model is used as a baseline and a Random Forest classifier is used as the main nonlinear model. The workflow includes preprocessing, categorical encoding, model training, evaluation and visualisation. The repository also documents how the prototype can be upgraded using public GTFS and GTFS-Realtime data.

1. Introduction

Smart cities increasingly use data-driven systems to improve transport planning and passenger information. Public transportation is influenced by recurring factors such as peak-hour traffic, passenger demand, weather, road conditions, route structure and previous delay propagation. Rural services may experience different reliability constraints because of longer distances, lower service frequency and less redundant transport infrastructure. A predictive system can help identify trips with a higher risk of delay before or during operation.

2. Problem Statement

The project addresses the problem: Can a supervised machine-learning model classify the expected delay level of a public transit trip using information available before or near the trip? The proposed system considers both urban and rural area types and produces an interpretable delay-risk category rather than claiming an exact arrival time.

3. Objectives

1. Study factors associated with public transit delay.
2. Represent urban and rural transit conditions in a common dataset.
3. Apply data preprocessing and feature engineering in Python.
4. Train supervised classification models.
5. Compare a linear baseline with a Random Forest model.
6. Evaluate predictions using accuracy, precision, recall and F1-score.
7. Create a reproducible GitHub-ready project.
8. Document a pathway for replacing demo data with public GTFS/GTFS-Realtime observations.

4. Dataset and Data Governance

The repository includes a synthetic demonstration dataset so the code can be executed immediately. This is explicitly labelled as synthetic and is not presented as observed transit data. For a final college analysis based on secondary public data, the recommended workflow is to use public GTFS schedule data and GTFS-Realtime Trip Updates/StopTimeUpdates. GTFS describes standard schedule information such as routes, trips, stops and stop times, while GTFS-Realtime can provide trip updates, vehicle positions, service alerts and delay/schedule-deviation values. A Pune GTFS case-study repository also documents the use of Pune transit data. The final submission should record the exact public source, date range, extraction date, license and

preprocessing steps.

5. Feature Description

area_type – Urban or Rural.

route_type – Local Bus, Express Bus or Feeder Bus.

hour – scheduled hour of the trip.

day_of_week – day index.

weather – broad weather category.

rainfall_mm – rainfall intensity.

traffic_index – congestion indicator from 0 to 100.

passenger_load_pct – estimated vehicle occupancy/load.

scheduled_headway_min – scheduled gap between services.

stop_count – number of stops in the trip/segment.

distance_km – route/segment distance.

road_condition – Good, Average or Poor.

previous_delay_min – prior delay available to the predictor.

special_event – indicator for a special event.

Target: delay_category – On Time/Minor, Moderate Delay or Major Delay.

6. Methodology

The data is divided into training and test sets using an 80:20 stratified split. Numeric variables are median-imputed and standardised; categorical variables are imputed and one-hot encoded. Logistic Regression provides a linear baseline. Random Forest is used as the primary model because it can represent nonlinear relationships and interactions without requiring a linear decision boundary. Class weighting is enabled to reduce the effect of class imbalance.

7. Evaluation

On the included demonstration dataset, Logistic Regression achieved an accuracy of 0.629, while the Random Forest model achieved an accuracy of 0.696. The Random Forest macro-average F1-score was 0.467. These values describe only the synthetic demonstration dataset and must not be interpreted as evidence of real-world transit performance. The confusion matrix is included in the repository to show where the classifier confuses adjacent delay categories.

8. Smart-City Applications

The proposed system can support passenger-information systems, operational control rooms, route planning, timetable adjustment, fleet allocation and proactive communication. A city could use predicted delay risk to flag services requiring attention. In rural areas, delay alerts may be particularly useful where service frequency is low and passengers have fewer alternative routes.

9. Limitations

The demo dataset is synthetic. Real deployment requires historical observed delays and careful prevention of target leakage. Public GTFS schedule data alone does not provide actual delay; a realtime or historical observed source is needed to construct the target. Weather, traffic and event data should be time-aligned with each trip. Model performance can change across cities, routes, seasons and operating conditions. The prototype predicts a category, not an exact arrival time.

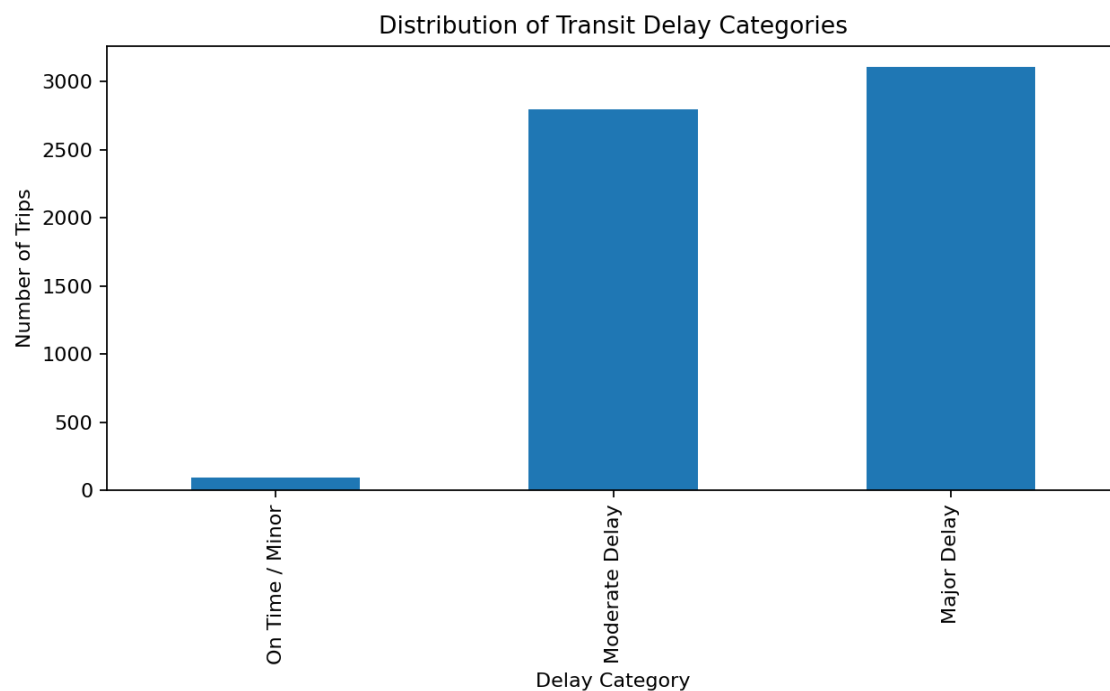
10. Future Scope

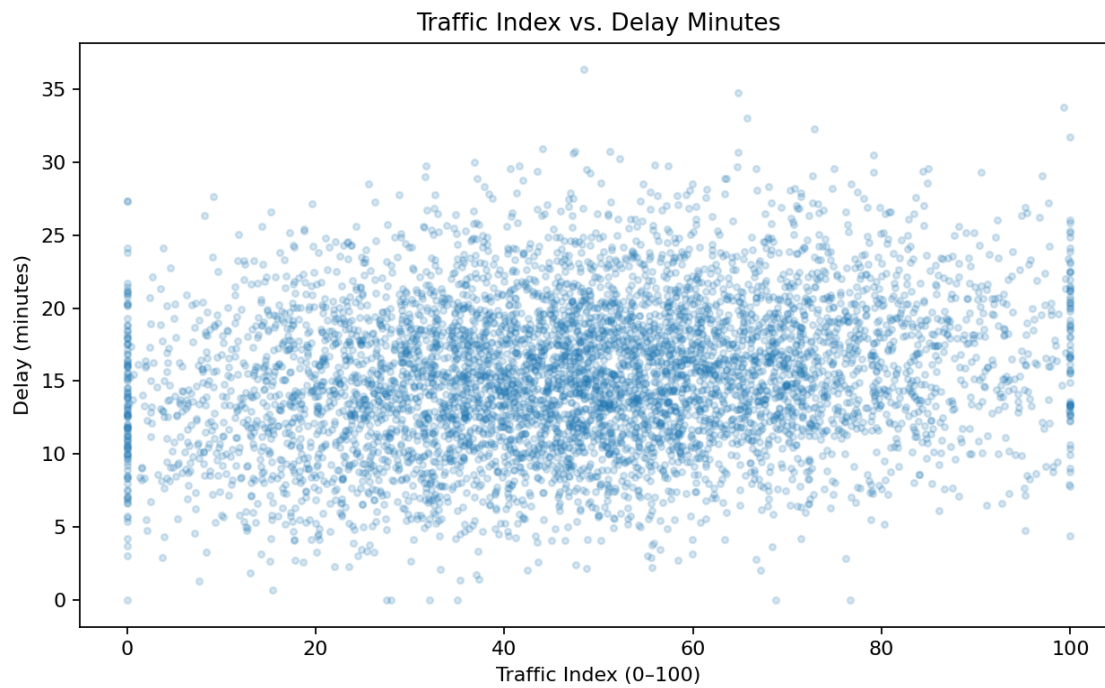
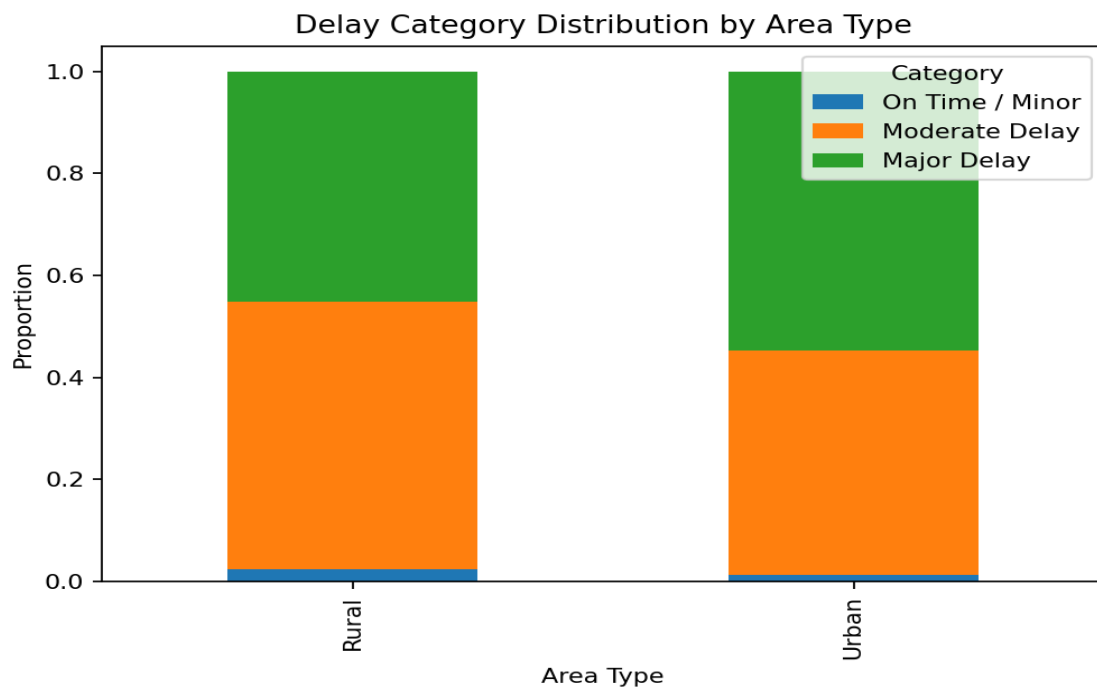
Future work can integrate live GTFS-Realtime feeds, historical weather, road traffic, holiday calendars and geospatial information. Regression models could predict delay minutes or ETA directly. Time-series or gradient-boosting approaches could be compared with Random Forest. A dashboard could show route-level risk, urban/rural comparisons and predicted delay categories. A production system should include model monitoring, drift detection, data validation and privacy/security controls.

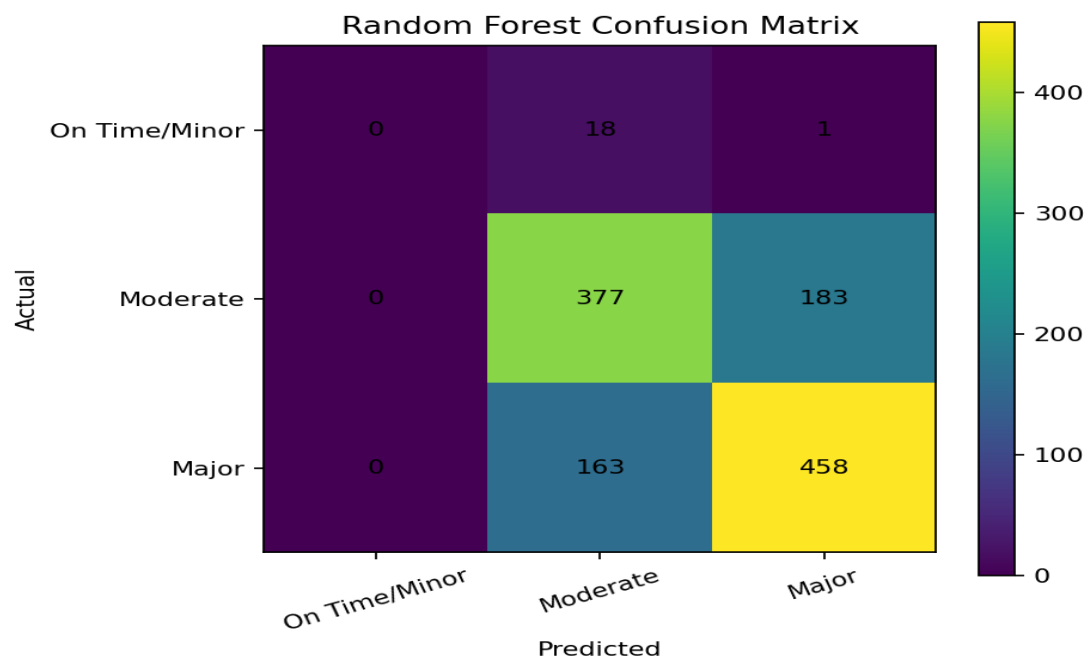
11. Conclusion

This project demonstrates a complete supervised-learning workflow for a smart-city public transit problem. It combines transport operations, environmental and temporal variables to classify delay risk. The project is intentionally reproducible and transparent about its demonstration data. The next research step is to replace the demo CSV with a documented public secondary dataset containing actual schedule and delay observations, then retrain and report results from that real dataset.

12. Results Figures







References / Data Sources

- GTFS Overview – <https://gtfs.org/documentation/overview/>
- GTFS Realtime Reference – <https://gtfs.org/documentation/realtime/reference/>
- Google GTFS-Realtime Overview – <https://developers.google.com/transit/gtfs-realtime>
- Pune GTFS Project – <https://github.com/charvibannur/GTFS-Project>
- Delhi Bus Transit Data Enhanced for ML – <https://www.kaggle.com/datasets/adarshdubey001/delhi-bus-transit-data-enhanced-for-ml>